JS - Day-7 Hands On – Chintha Ramakrishna

Problem 1 :-

Create a Student Marks Analyzer program that stores marks in an array, calculates total and average using array methods, and displays whether the student passed or failed using ES6 modular JavaScript.

Code :-

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<title>Student Marks Analyzer</title>



<style>

body{

    font-family: Arial;

    background:#d68306;

    display:flex;

    justify-content:center;

    align-items:center;

    height:100vh;

}



.container{

    background:rgb(226, 215, 65);
```

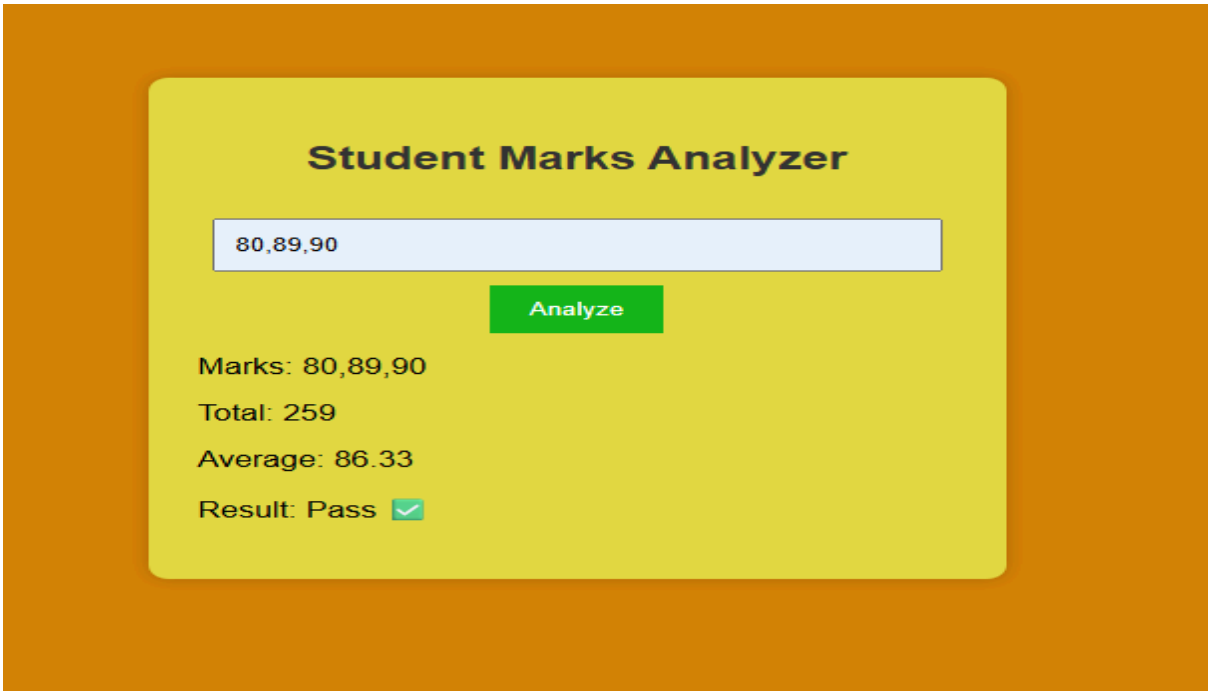
```
    padding:25px;
    border-radius:10px;
    width:380px;
    text-align:center;
    box-shadow:0 0 10px rgba(0,0,0,0.2);
}

h2{
    color:#333;
}

input{
    width:90%;
    padding:10px;
    margin:10px 0;
}

button{
    padding:10px 20px;
    background:#17b61d;
    color:white;
    border:none;
    cursor:pointer;
}
```

```
button:hover{
    background:#45a049;
}

.result{
    margin-top:15px;
    text-align:left;
}
</style>
</head>

<body>

<div class="container">

<h2>Student Marks Analyzer</h2>

<input type="text" id="marksInput" placeholder="Enter marks (e.g
80,70,90)">
<button onclick="analyzeMarks()">Analyze</button>

<div class="result" id="output"></div>

</div>
```

```
<script type="module">

const analyzeMarks = () => {

    const input = document.getElementById("marksInput").value;

    // Convert input string to array of numbers using map()
    const marks = input.split(",").map(num => Number(num.trim()));

    // Calculate total using reduce()
    const total = marks.reduce((sum,m)=> sum + m ,0);

    const average = marks.length ? (total / marks.length) : 0;

    const result = average >= 40 ? "Pass ✅" : "Fail ❌";
    // Display output using template literals.
    document.getElementById("output").innerHTML = `
        <p>Marks: ${marks}</p>
        <p>Total: ${total}</p>
        <p>Average: ${average.toFixed(2)}</p>
        <p>Result: ${result}</p>
    `;
};
```

```
// make function global

window.analyzeMarks = analyzeMarks;



</script>



</body>

</html>
```

Output :-



Explanation :-

1. Student marks are stored inside an array so we can process multiple values easily.
2. We use the reduce() method to calculate the total marks by adding all elements.
3. We use the map() method to transform marks if needed (for example, grading or formatting).
4. The average is calculated by dividing total marks by the number of subjects and we decide pass/fail based on that average.

5. Using ES6 features like arrow functions, template literals, let/const, and modules makes the code clean, modern, and reusable.

Problem 2 :-

Create a Product Cart Summary program that stores products (name, price, quantity) in an array, calculates the total cart value using map() and reduce(), and exports the calculation function using ES6 modules.

Code :-

Index.html :-

```
<!DOCTYPE html>
<html>
<head>
<title>Product Cart Summary</title>

<style>
body{
font-family: Arial;
background:#f2f2f2;
display:flex;
justify-content:center;
align-items:center;
height:100vh;
}
.box{
background:white;
padding:20px;
border-radius:10px;
width:350px;
box-shadow:0 0 10px gray;
}
h2{text-align:center;}
pre{
background:#eee;
padding:10px;
border-radius:5px;
}
</style>
</head>
<body>
```

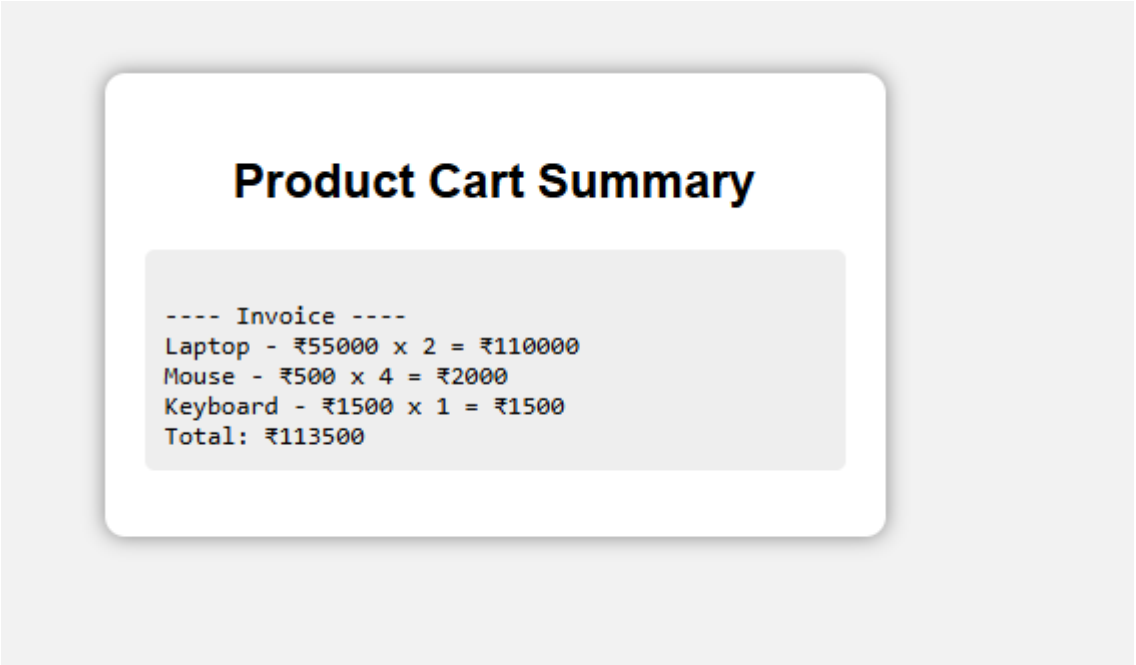
```html
<div class="box">
<h2>Product Cart Summary</h2>
<pre id="output"></pre>
</div>

<!-- module script -->
<script type="module">

import { calculateInvoice } from "./cartModule.js";

const cart = [
{name:"Laptop", price:55000, qty:2},
{name:"Mouse", price:500, qty:4},
{name:"Keyboard", price:1500, qty:1}
];

const invoice = calculateInvoice(cart);

document.getElementById("output").textContent = invoice;

</script>

</body>
</html>
```

Cart.js :-

```js
// cartModule.js

// calculation function
export const calculateInvoice = (cart) => {
    let total = 0;

    const lines = cart.map(item => {
        const subtotal = item.price * item.qty;
        total += subtotal;
        return `${item.name} - ₹${item.price} x ${item.qty} =
₹${subtotal}`;
    });

    return `
---- Invoice ----
${lines.join("\n")}
```

```
Total: ₹${total}
`;
};
```

Output :-



Explanation :-

1. Product details are stored as objects inside an array so multiple items can be managed easily.
2. The map() method is used to calculate each product's subtotal (price × quantity).
3. The reduce() method is used to add all subtotals and find the total cart value.
4. Template literals help display the invoice in a clean formatted way with dynamic values.
5. By exporting the calculation function using ES6 modules, the logic becomes reusable and keeps code organized.

Problem 3 :-

Create a Weather Data Fetcher program that retrieves weather information from a public API using fetch(), first with Promises and then using async/await, while handling errors properly and displaying a formatted weather report.

Code :-

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<meta charset="UTF-8">
<title>Weather Fetcher</title>
</head>
<body>

<h2>Weather Report</h2>
<button onclick="getWeatherPromise()">Fetch (Promise)</button>
<button onclick="getWeatherAsync()">Fetch (Async/Await)</button>

<pre id="output"></pre>

<script>
// API URL (Hyderabad example)
const url =
"https://api.open-meteo.com/v1/forecast?latitude=17.38&longitude=78.48&
current_weather=true";

const display = (text) => {
  document.getElementById("output").innerText = text;
};


// PROMISE VERSION

const getWeatherPromise = () => {
  fetch(url)
    .then(res => {
      if (!res.ok) throw new Error("Network error");
      return res.json();
    })
    .then(data => {
      const w = data.current_weather;

      const report = `
--- WEATHER REPORT (Promise) ---
Temperature : ${w.temperature}°C
Wind Speed  : ${w.windspeed} km/h
Time        : ${w.time}
`;
      display(report);
    })
    .catch(err => display("Error: " + err.message));
};
```

```
//  ASYNC / AWAIT VERSION

const getWeatherAsync = async () => {
  try {
    const res = await fetch(url);

    if (!res.ok) throw new Error("Network error");

    const data = await res.json();
    const w = data.current_weather;

    const report = `
--- WEATHER REPORT (Async/Await) ---
Temperature : ${w.temperature}°C
Wind Speed  : ${w.windspeed} km/h
Time        : ${w.time}
`;
    display(report);

  } catch (err) {
    display("Error: " + err.message);
  }
};
</script>

</body>
</html>
```

Output :-

Weather Report

Fetch (Promise) | Fetch (Async/Await)

```
--- WEATHER REPORT (Promise) ---
Temperature : 28.2°C
Wind Speed  : 2.7 km/h
Time        : 2026-02-24T12:30
```

Explanation :-

1. The application uses the fetch() function to request weather data from a public API asynchronously.

2. In the Promise version, we handle the response using .then() and .catch() to process data and manage errors.
3. In the modern approach, async/await makes asynchronous code easier to read and write like normal synchronous code.
4. try/catch is used to handle errors such as network issues or invalid API responses.
5. Template literals and arrow functions help display the weather report in a clean, modern ES6 format.

Problem 4 :-

Create a Task Manager with Async Storage program that stores tasks in an array and demonstrates callback → Promise → async/await approaches using ES6 modules.

Code :-

```
<!DOCTYPE html>
<html>
<head>
<title>Task Manager Async</title>

<style>
body{
font-family:Arial;
background:#f2f2f2;
display:flex;
justify-content:center;
align-items:center;
height:100vh;
}
.box{
background:white;
padding:20px;
border-radius:10px;
width:360px;
box-shadow:0 0 10px gray;
}
pre{background:#eee;padding:10px;border-radius:5px;}
button{margin:5px;}
</style>
</head>
<body>
```

```html
<div class="box">
<h2>Task Manager</h2>

<button id="run">Run Demo</button>
<pre id="output"></pre>
</div>

<script type="module">
import { addTask, deleteTask, listTasks } from "./taskModule.js";

const output = document.getElementById("output");

document.getElementById("run").onclick = async () => {

    let text = "";

    text += `${await addTask("Learn JavaScript")}\n`;
    text += `${await addTask("Practice DSA")}\n`;
    text += `${await listTasks()}\n\n`;

    text += `${await deleteTask("Learn JavaScript")}\n`;
    text += `${await listTasks()}`;

    output.textContent = text;
};
</script>

</body>
</html>
```

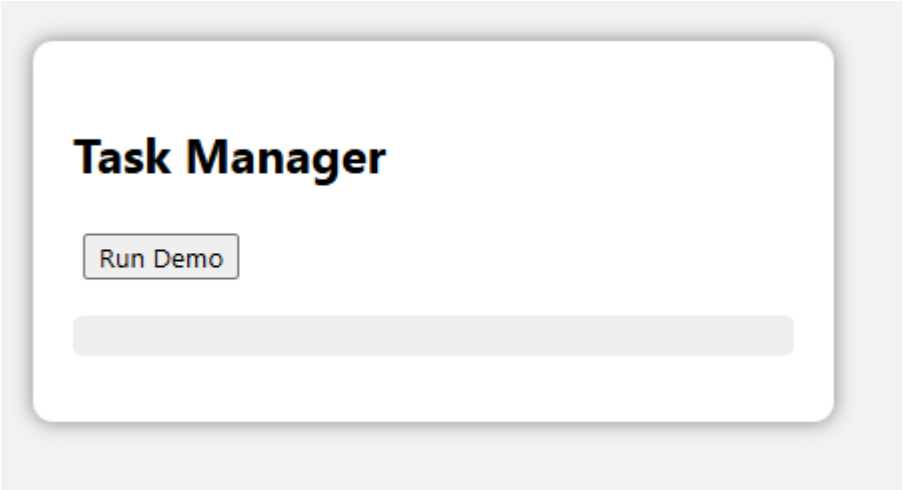
Taskmodule.js :-

```javascript
// taskModule.js

let tasks = [];

/* ========================
   1️⃣ Callback Version
======================== */
export const addTaskCallback = (task, callback) => {
    setTimeout(() => {
        tasks.push(task);
        callback(`Task Added: ${task}`);
    }, 500);
```

```
};

export const deleteTaskCallback = (task, callback) => {
    setTimeout(() => {
        tasks = tasks.filter(t => t !== task);
        callback(`Task Deleted: ${task}`);
    }, 500);
};

export const listTasksCallback = (callback) => {
    setTimeout(() => {
        callback(`Tasks:\n${tasks.join("\n")}`);
    }, 500);
};


/* ========================
   2⃣ Promise Version
======================== */
export const addTaskPromise = (task) =>
    new Promise(resolve => {
        setTimeout(() => {
            tasks.push(task);
            resolve(`Task Added: ${task}`);
        }, 500);
    });

export const deleteTaskPromise = (task) =>
    new Promise(resolve => {
        setTimeout(() => {
            tasks = tasks.filter(t => t !== task);
            resolve(`Task Deleted: ${task}`);
        }, 500);
    });

export const listTasksPromise = () =>
    new Promise(resolve => {
        setTimeout(() => {
            resolve(`Tasks:\n${tasks.join("\n")}`);
        }, 500);
    });
```

```
/* ==========================
   3️⃣ Async/Await Version
========================== */
export const addTask = async (task) => {
    return await addTaskPromise(task);
};

export const deleteTask = async (task) => {
    return await deleteTaskPromise(task);
};

export const listTasks = async () => {
    return await listTasksPromise();
};
```

Output :-



Explanation :-

1. **Tasks are stored in an array and managed using functions like addTask(), deleteTask(), and listTasks().**
2. **setTimeout() is used to simulate asynchronous storage and the first version works with callbacks.**
3. **The same logic is converted into Promises to provide better control over asynchronous flow.**
4. **Finally, the code is refactored using async/await, making it cleaner and easier to understand with proper error handling.**
5. **Using ES6 modules, arrow functions, let/const, and template literals helps organize code and follow modern JavaScript practices.**